FRAMEWORK CYPRESS

Aluna: Giseli Souza de Matos

PIRÂMIDE DE TESTES

- Testes End-to-End - E2E

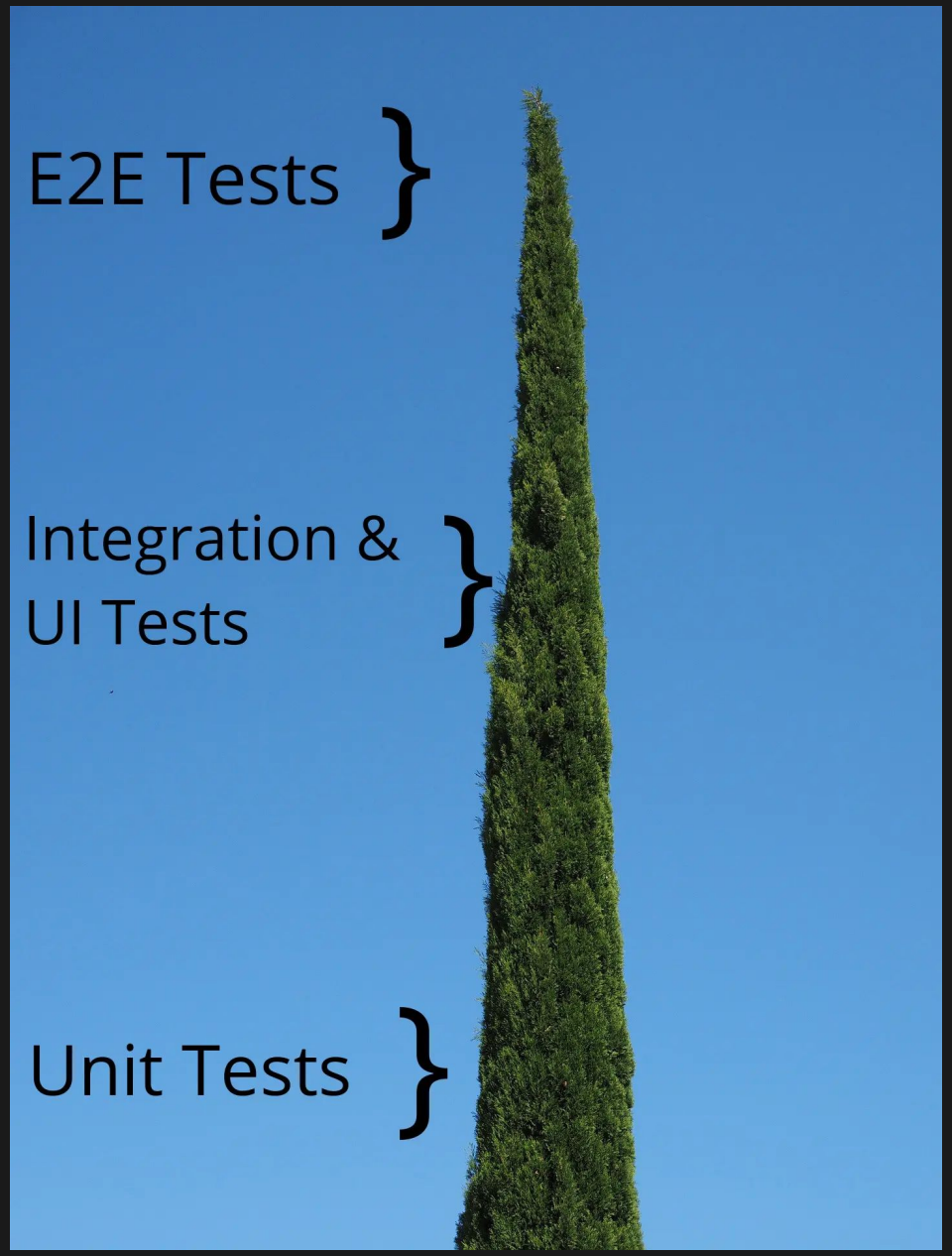
Simulam a jornada completa do usuário final, do início ao fim (ex: abrir o site, realizar cadastro, fazer login e enviar uma mensagem).

- Testes de Componentes:

Validam unidades isoladas da interface ou lógica (ex: um botão de cadastro, um formulário ou uma função de cálculo) sem depender do sistema completo.

- Testes Unitários:

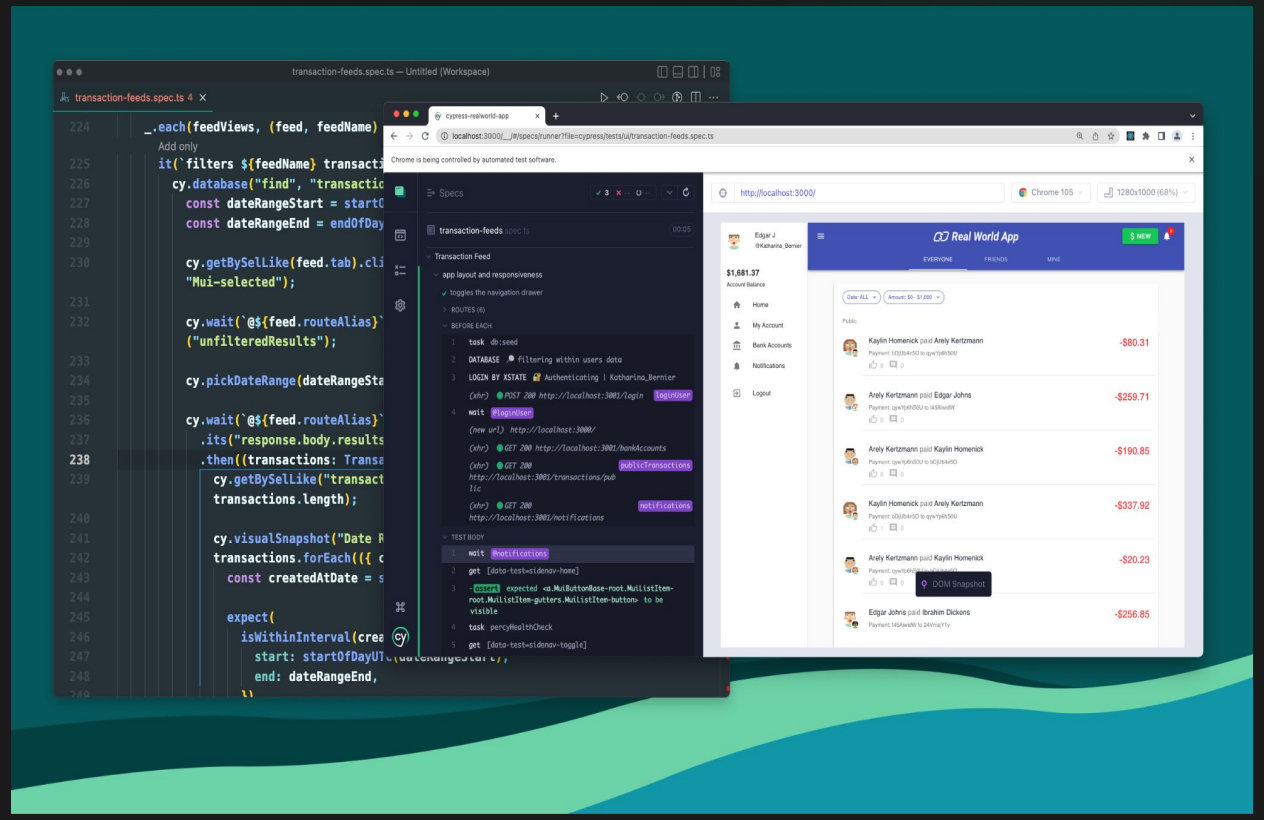
É o processo de testar a menor unidade funcional de um software de forma isolada (como uma única função, método ou classe).



INTRODUÇÃO AO FRAMEWORK

Cypress é um framework de testes de ponta a ponta (E2E) baseado inteiramente em **JavaScript**. Diferente de outras ferramentas, ele roda diretamente dentro do navegador, permitindo uma interação nativa com a aplicação.

- Execução em tempo real.
- Foco em aplicações web modernas (React, Vue, Angular).
- Arquitetura sem Selenium ou drivers externos.



PRÓS E CONTRAS

Vantagens

- Fácil de aprender (principalmente para quem já usa JS).
- Documentação excelente.
- Setup simples e rápido.
- Debug visual (mostra exatamente o que está acontecendo).
- Execução mais rápida que Selenium.
- Esperas automáticas evitam testes instáveis.

Desvantagens

- Suporte restrito a JS/TS apenas.
- Sem suporte nativo para múltiplas abas do navegador.
- Não é ideal para testes de performance pesada ou carga (back end).
- Consome mais recursos da máquina.
- Não é ideal para testes mobile nativos.

PRINCIPAIS CARACTERÍSTICAS

Snapshots

Captura de screenshots e vídeos durante a execução dos testes para análise posterior.

Flake Resistance

Algoritmos internos que garantem que os testes não falhem por problemas de latência de rede.

Stubbing

Controle total sobre requisições de rede, permitindo simular respostas de APIs e estados de erro.

LICENÇA E GESTÃO

MIT

Licença Open Source

Desenvolvimento

O Cypress é mantido pela empresa **Cypress.io**, que oferece um modelo "Freemium". O Test Runner (CLI) é totalmente gratuito, enquanto o Cypress Cloud (para dashboards e paralelismo) é um serviço proprietário pago.

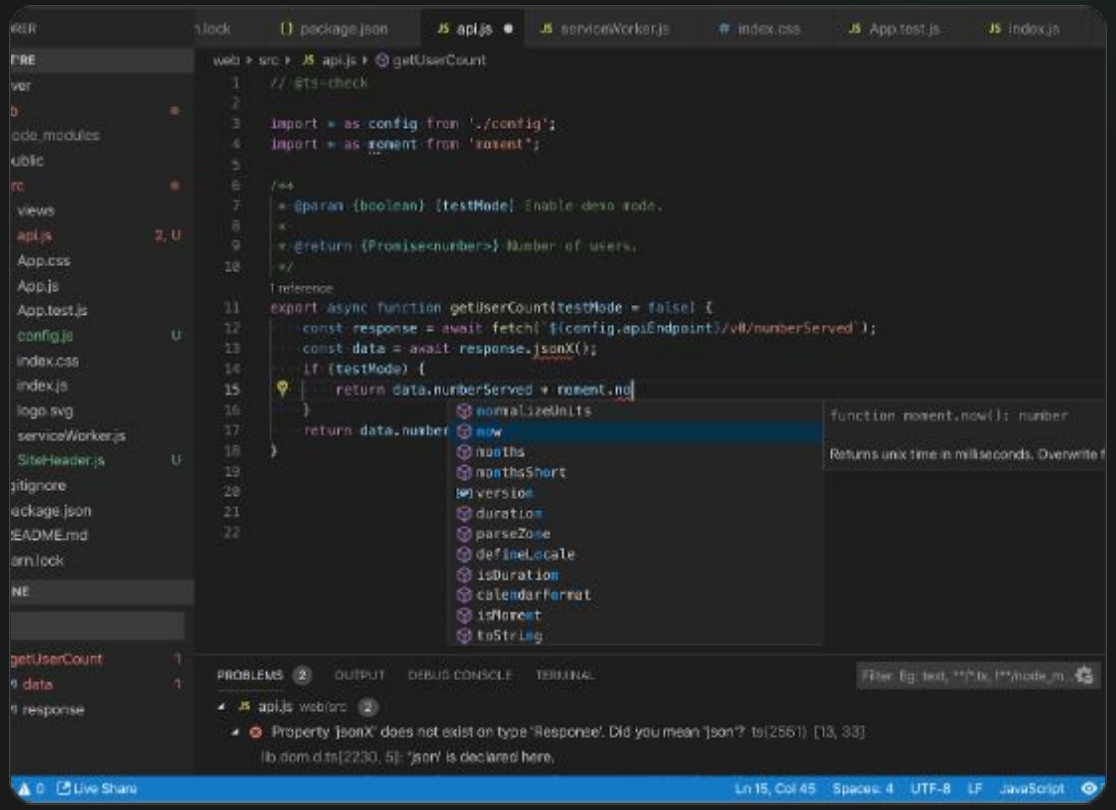
A comunidade é uma das maiores do ecossistema QA, com milhões de downloads semanais.

AMBIENTE E INTEGRAÇÃO

Categoria	Tecnologias Suportadas / Exemplo	Nível de Suporte
Servidores Web	Node.js, Python (Django), PHP (Laravel), Java (Spring), Ruby on Rails	Totalmente Independente
Ferramentas CI/CD	Jenkins, GitHub Actions, GitLab CI, CircleCI, Azure Pipelines	Nativo via CLI
Navegadores	Chrome, Edge, Firefox, Electron, Safari (WebKit)	Multibrowser Nativo

REQUISITOS E CONFIGURAÇÃO

- **Node.js:** Versão 14.x ou superior recomendada.
- **Hardware:** Mínimo 4GB de RAM (8GB recomendado para CI).
- **Package Manager:** npm, yarn ou pnpm instalados.
- **Sistemas Operacionais:** Windows (7+), macOS (10.9+) e Linux (Ubuntu/CentOS).



INSTALAÇÃO E EXECUÇÃO

Passo 1

Instale via npm:
npm install cypress
--save-dev

Passo 2

Crie seu arqivo
teste.cy.js.

Passo 3

Abra o framework:
npx cypress open

Passo 4

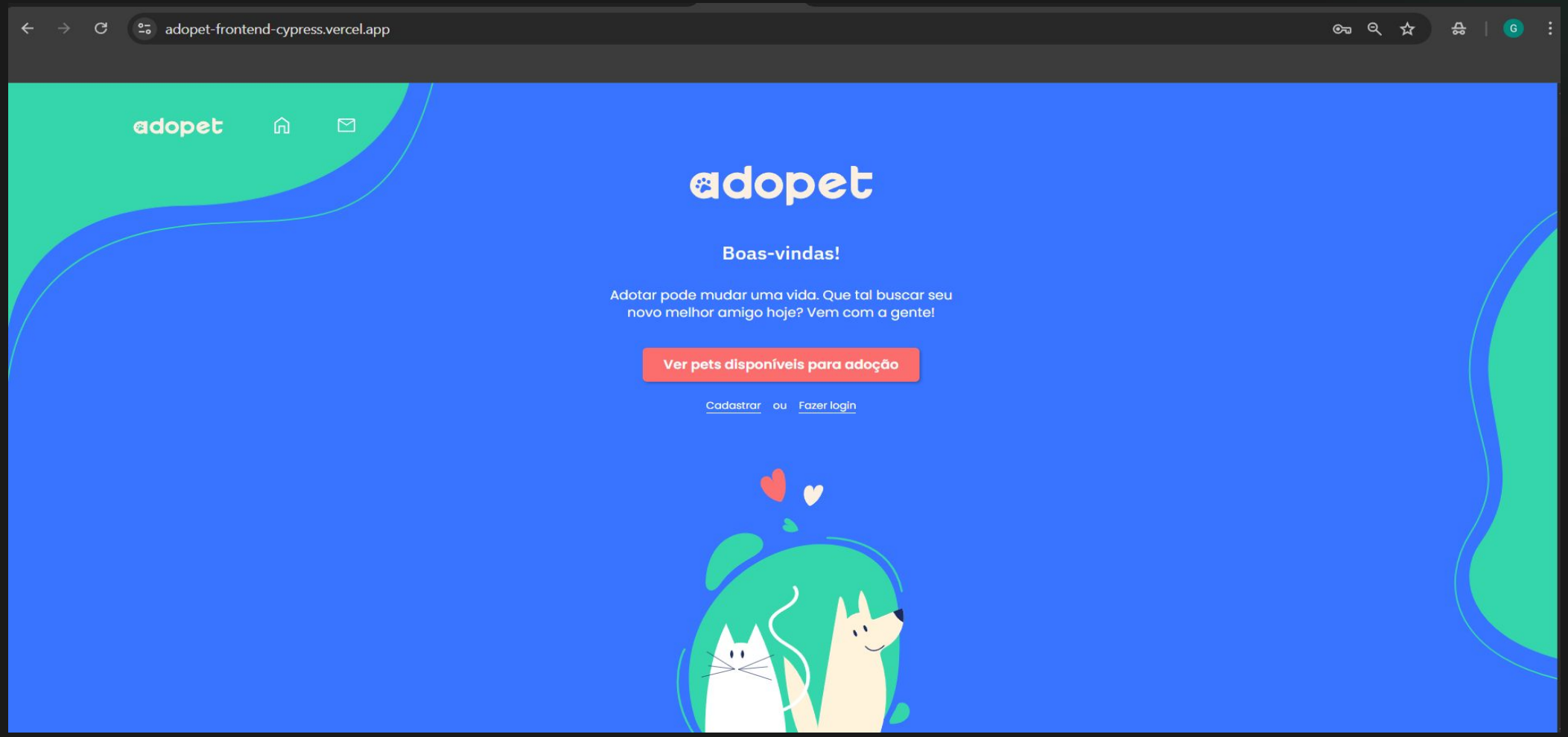
Configure o E2E e
escolha o Navegador.

Passo 3

Execute no terminal:
npx cypress run

APLICAÇÃO EXEMPLO

https://adopet-frontend-cypress.vercel.app/ https://github.com/alura-cursos/adopet-frontend-cypress



FLUXO DE CADASTRO

cadastro-massa.cy.js U

cypress > e2e > cadastro-massa.cy.js > ...

```
import { usuarios } from '../fixtures/usuarios.json'

describe('Página de cadastro', () => {
    beforeEach(() => {
        cy.visit('https://adopet-frontend-cypress.vercel.app');
        cy.get('[data-test="register-button"]').click();
    })
    usuarios.forEach(usuario => {
        it('Deve preencher os campos do formulário corretamente para cadastrar um novo usuário', () => {
            cy.get('[data-test="input-name"]').clear().type(usuario.name);
            cy.get('[data-test="input-email"]').clear().type(usuario.email);
            cy.get('[data-test="input-password"]').clear().type(usuario.password);
            cy.get('[data-test="input-confirm-password"]').clear().type(usuario.password);
            cy.get('[data-test="submit-button"]').click();
        })
    })
})
```

Cenário de Sucesso: Validamos a criação de conta preenchendo Nome, E-mail e Senha. Utilizamos comandos customizados (cy.cadastrar) para reaproveitamento de código.

Cenário de Falha: Validamos as mensagens de erro ("É necessário informar um endereço de email") quando os campos são deixados em branco.

Data Driven Testing: No arquivo cadastro-massa.cy.js, demonstramos como rodar o mesmo teste para múltiplos usuários através de um arquivo JSON (fixtures).

FLUXO DE LOGIN

Validação de Regras: Testamos se o sistema bloqueia senhas fracas ou e-mails fora do padrão.

Mocking/Stubbing: No arquivo login-incorreto.cy.js, usamos o cy.intercept para simular um erro 400 da API, testando o comportamento do frontend mesmo quando o servidor falha.

login-incorreto.cy.js U

cypress > e2e > login-incorreto.cy.js > ...

```
1  describe('Página de login', () => {
2
3    beforeEach(() => {
4      cy.visit('https://adopet-frontend-cypress.vercel.app');
5      cy.get('[data-test="login-button"]').click();
6      cy.intercept('POST', 'https://adopet-api-i8qu.onrender.com/adotante/login',{
7        statusCode:400, }).as('stubPost')
8
9    })
10
11       it('Deve preencher os campos do login incorretamente e exibir mensagens de erro ao usuário na página', () => {
12         cy.get('[data-test="input-loginEmail"]').type('anamail');
13         cy.get('[data-test="input-loginPassword"]').type('123');
14         cy.get('[data-test="submit-button"]').click();
15         cy.contains('Por favor, verifique o email digitado').should('be.visible')
16         cy.contains('A senha deve conter pelo menos uma letra maiúscula, um número e ter entre 6 e 15 caracteres')
17         .should('be.visible')
18
19       })
20
21       it('Deve falhar mesmo que os campos sejam preenchidos corretamente', () => {
22         cy.login('ana@email.com', 'Senha123')
23         cy.wait('@stubPost')
24         cy.contains('Falha no login. Consulte suas credenciais.').should('be.visible')
25       })
26
27
28 })
```

TESTE DE API

```
api-mensagens.cy.js U ×
cypress > e2e > api-mensagens.cy.js > ...
1   describe('Api Adopet', () => {
2       const authorization = `Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1ODhmNTNhOS1kYzVml
3
4   it('Mensagens da API', () => {
5           cy.request({
6            method: 'GET' ,
7            url: 'https://adopet-api-i8qu.onrender.com/mensagem/11643cd6-7112-415b-95d2-07904b0d1a1c',
8            headers: { authorization }
9           }).then((res) => {
10              expect(res.status).to.be.equal(200)
11          expect(res.body).is.not.empty
12          expect(res.body).to.have.property('msg')
13          })
14  })
15  })
```

No arquivo api-mensagens.cy.js, demonstramos como testar diretamente o endpoint de mensagens:

Enviamos um token de Authorization (JWT).

Validamos o Status Code 200.

Verificamos se o corpo da resposta (res.body) contém a propriedade msg e não está vazio.

DOCUMENTAÇÃO

- ScreenshotOnRunFailure: tira print automaticamente em casos de erros

- Video Recording: *video: true* para gravar as execuções.

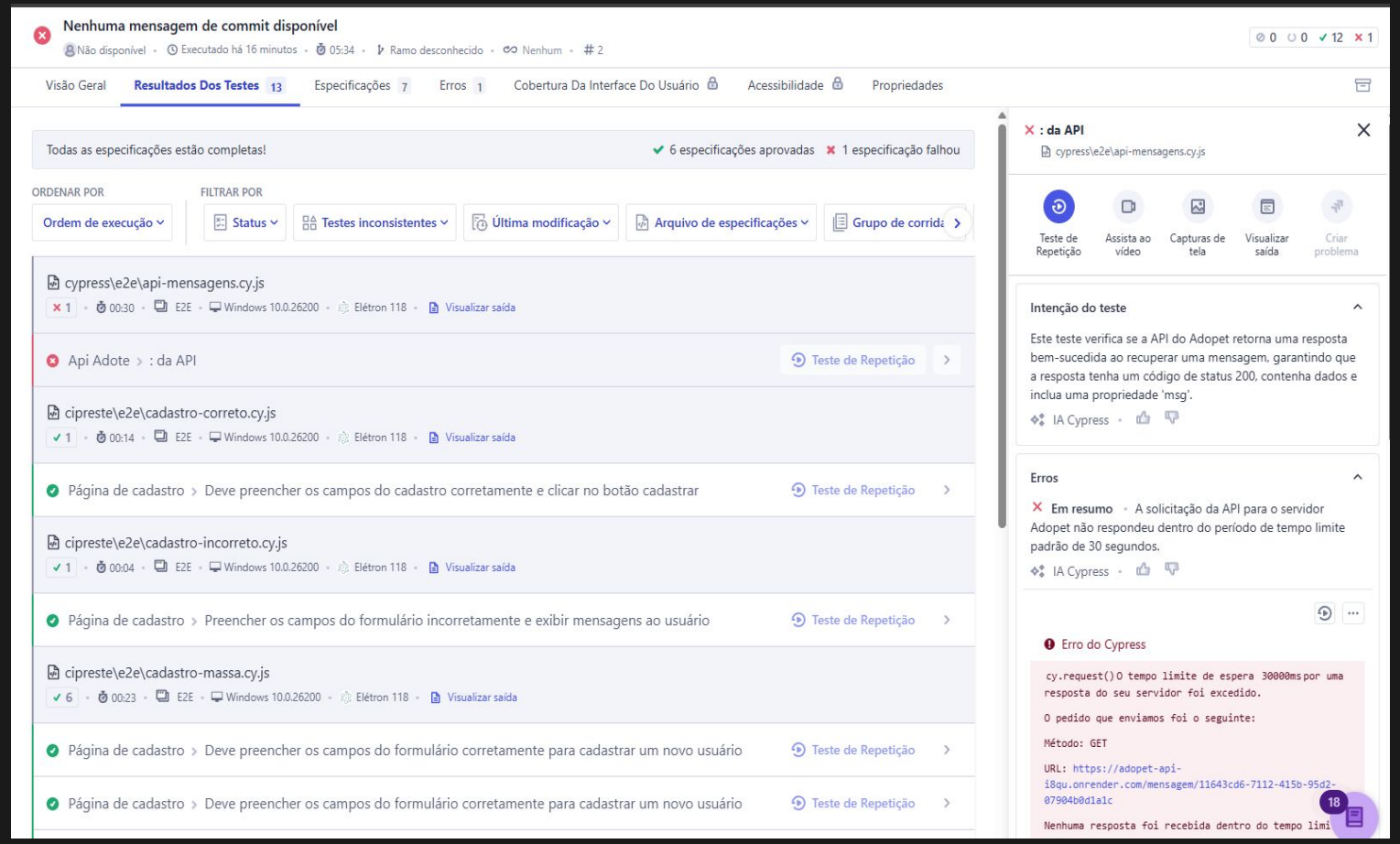
- Mochawesome Reporter: Gerador de relatórios HTML detalhados com timestamps, salvos em cypress/results.

cypress.config.js U | Api Adopet -- Mensagens da API (failed).png U

cypress.config.js > ...

```
1  const { defineConfig } = require("cypress");
2
3  module.exports = defineConfig({
4    projectId: "wk66nz",
5    e2e: {
6      setupNodeEvents(on, config) {
7        //
8      },
9      video: true,
10     screenshotOnRunFailure: true,
11     reporter: 'mochawesome',
12
13     reporterOptions: {
14       reportDir: 'cypress/results',
15       overwrite: true,
16       html: true,
17       json: false,
18       timestamp: "mmddyyyy_HHMMss" }
19   },
20 });
```

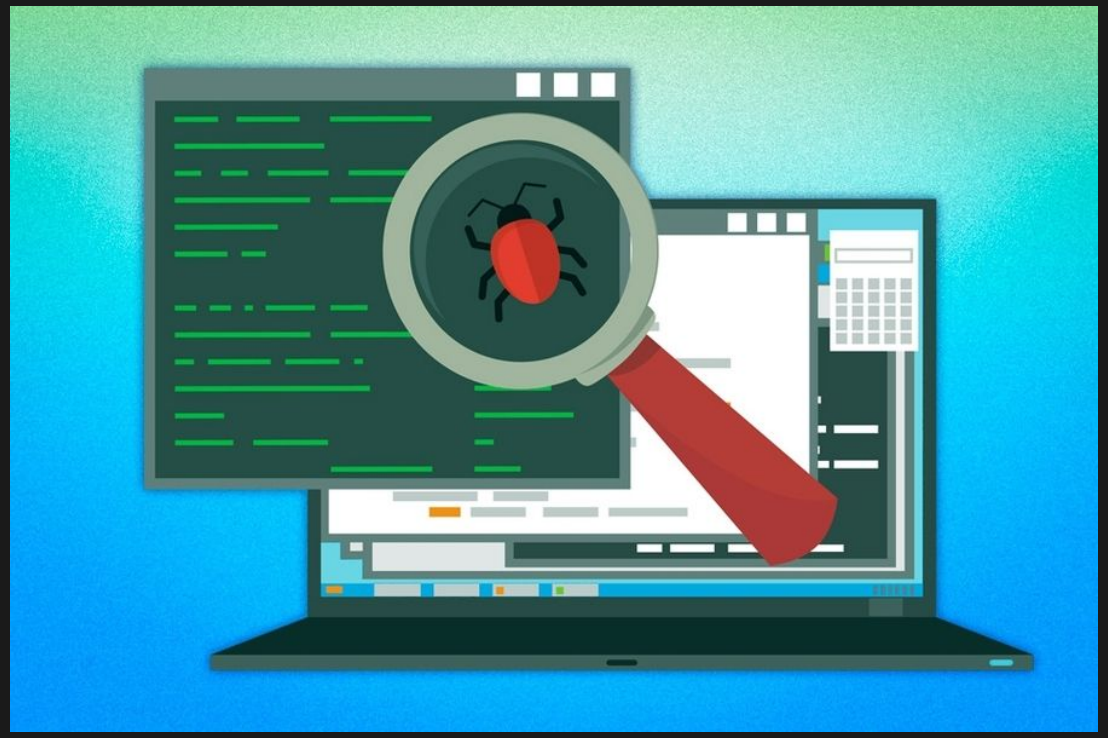
CYPRESS CLOUD



- Acesse o site Cypress Cloud.
- Faça login (pode usar sua conta do GitHub).
- Crie um novo projeto.
- No painel do Cypress Cloud, vá em Project Settings.
- Localize a seção Record Keys e copie a chave gerada.

```
npx cypress run --record --key SUA_CHAVE
```

CONCLUSÃO



- A curva de aprendizado é baixa e a qualidade da documentação oficial é excepcional.
- Regressão: Certeza de que novas atualizações não quebraram o cadastro.
- Confiança: Validação das regras de negócio (e-mail e senha).
- Documentação: Os scripts de teste servem como a especificação técnica do comportamento do site.

Obrigado pela atenção!